Intro to LLM reasoning

Sunay Joshi

Overview

- Question: how does OpenAI o1 work?
- Datasets
- Basic techniques (← test-time scaling)
- “Improve generator only”
- “Learn verifier only” (← test-time scaling)
- “Learn verifier + Improve generator”
- Probably will have lots of time for discussion (discuss DeepSeek-R1?)

Please interrupt if I say anything incorrect.

Datasets

- GSM8K: grade school math questions
- MATH: contest level math questions
- Split into train and test sets

- PRM800K: step-level human-annotated solutions for MATH dataset
- Surprising amount of human effort (through companies like Scale/SurgeAI)

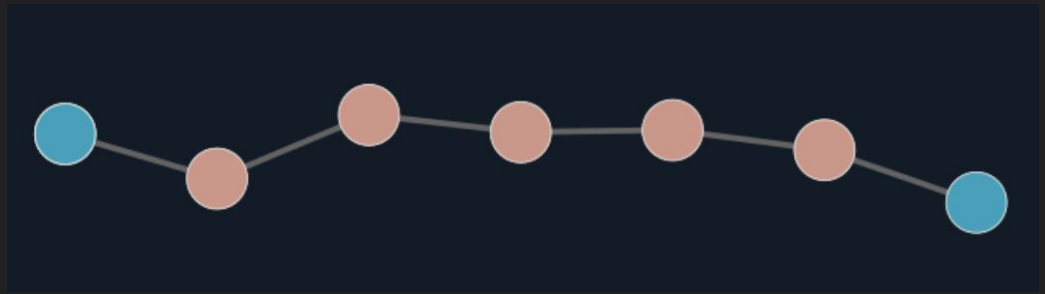
Basic techniques

- Start with a model finetuned on (Q, A) pairs from the train portion of the dataset (“SFT-only”)
- Recall: a Chain-of-Thought (CoT) is a sequence of reasoning steps
- Zero-shot CoT prompting (“Let’s think step by step”) (Wei et al., 2022)
- Few-shot CoT prompting (the above + in-context examples)
- Other prompting techniques
- Best-of-N sampling + majority vote / self-consistency
  - Test-time scaling
  - For hard problems, one should set N to be large
- But these have their limits

Notation (from Rush's slides)

- x; problem specification
- $z_{1:T} \in \mathcal{S}^T$; chain of thought (CoT) steps
- $y \in \mathcal{Y}$; final answer

$$z_{1:T} \sim p(\cdot|x)$$
$$y \sim p(\cdot|x, z_{1:T})$$



Improve generator (only)

The following is done at *train-time.*

The original LLM is called the generator. How can we improve it?

At iteration k:

1. Sample N CoTs from the current generator
2. Use labels to mark each answer as “correct” / “incorrect”
3. Finetune on (some of) these CoTs

Improve generator (only) (cont'd)

Singh et al., 2023 ("Beyond Human Data: Scaling Self-Training…", Google)

- ReST-EM algorithm (Reinforced Self-Training)
- Filter-out CoTs with incorrect answers (E-step)
- Finetune *base* model on these (improves stability) (M-step)

Shao et al., 2024 ("DeepSeekMath…", DeepSeek)

- GRPO algorithm (Group Relative Policy Optimization)
- Compute score for each CoT based on answer + format (use *all* CoTs)
- Finetune previous policy on these

(General: Liu et al., 2024 ("Rejection Sampling Improves Preference Optimization", Google).)

Extra details

(our notation: log p(y correct | x))

ReST-EM: use EM to maximize the log likelihood $\log p(O = 1 \mid \boldsymbol{x})$

for $i = 1$ *to* I **do**
  // Generate (E-step)
  Generate dataset $\mathcal{D}_i$ by sampling: $\mathcal{D}_i = \{ (\boldsymbol{x}^j, \boldsymbol{y}^j)|_{j=1}^N \text{ s.t. } \boldsymbol{x}^j \sim \mathcal{D},\ \boldsymbol{y}^j \sim p_\theta(\boldsymbol{y}|\boldsymbol{x}^j) \}$
  Annotate $\mathcal{D}_i$ with the reward $r(\boldsymbol{x}, \boldsymbol{y})$.
  // Improve (M-step)
  while *reward improves on* $\mathcal{D}_{val}$ **do**
    Optimise θ to maximize objective: $J(\theta) = \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim\mathcal{D}_i} [r(\boldsymbol{x}, \boldsymbol{y}) \log p_\theta(\boldsymbol{y}|\boldsymbol{x})]$
  end

Extra details

GRPO: same as PPO (clipping + KL-regularization). Use advantages of all CoTs.

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right),$$

(our notation: o_i = CoT, q = x)

$$A_i = \frac{r_i - mean(\{r_1, r_2, \cdots, r_G\})}{std(\{r_1, r_2, \cdots, r_G\})}.$$

Reward models

What if, at *test-time,* you could score the correctness of each CoT?

This is called a *verifier,* or a *reward model* (RM).

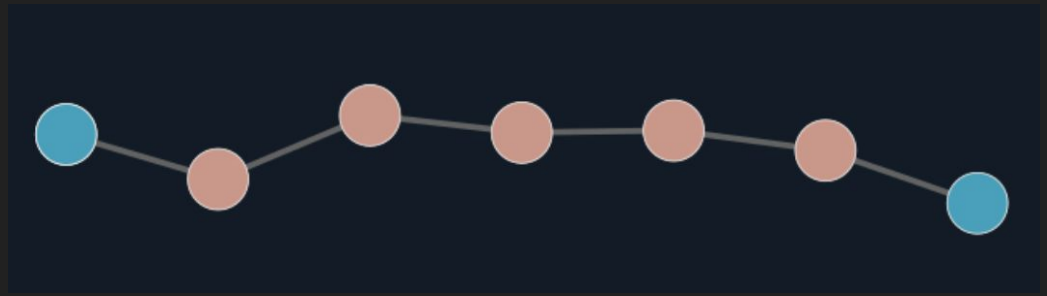
(Now we have: generator + RM. We don’t touch the generator.)

Two kinds of RMs: *outcome* RMs and *process* RMs.

Outcome RM (ORM): provide score for the final answer. “Sparse”, instance-level

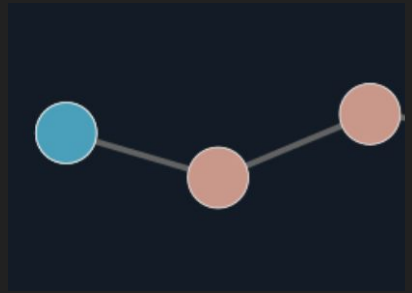
Process RM (PRM): provide score for *each step* of the CoT. “Dense”, step-level

Notation (from Rush's slides)



ORM → a score $r(x, y)$

$$r : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$$



PRM → a score $r_t(x, z_{1:t})$

$$r_t : \mathcal{X} \times \mathcal{Z}^t \to \mathbb{R}$$

Reward models

The terminology comes from Reinforcement Learning (RL).

In RL, an agent is at a state, takes an action, collects a reward, and repeats.

Goal: maximize the total reward.

By taking actions, the agent receives feedback from the environment (“reward signals”), and learns good state-action pairs.

If feedback is sparse,
Can’t attribute the credit,
Takes longer to learn.

Analogy: agent = LLM, state = partial CoT, action = next step, reward = OR/PR

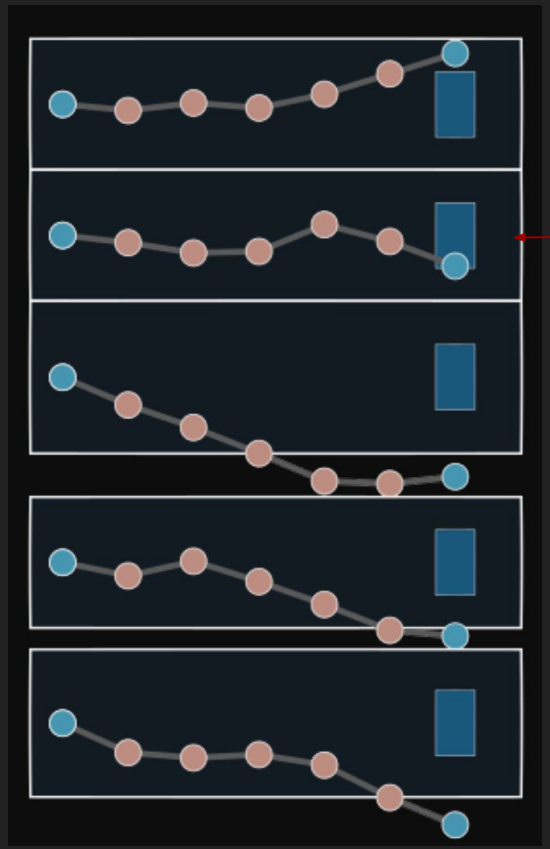
Learn ORM (only)

Suppose you learned an ORM. How to use it at test-time?

1. Sample N CoTs from the generator
2. Score each CoT using the ORM
3. Return the answer of the highest-scoring CoT
   (Or vote among top-K CoTs)

(Natural concern: for hard problems, you might not sample *any* correct CoTs! → PRMs)

Learn ORM (only) (cont'd)



(From Sasha Rush's slides)

How to learn ORM?

Cobbe et al., 2021 (“Training Verifiers to Solve Math Word Problems”, Open AI)

- Sample 100 CoTs for each training problem + label each
- Train ORM to predict Pr(correct | CoT) for 1 epoch
- The ORM is another LLM + a scalar head
- (The ORM actually makes predictions at the token-level, but aggregates them)

Sometimes automatic (i.e. for structured math, programming).

Learn PRM (only), cont’d

Suppose you learned a PRM. How to use it at *test-time?*

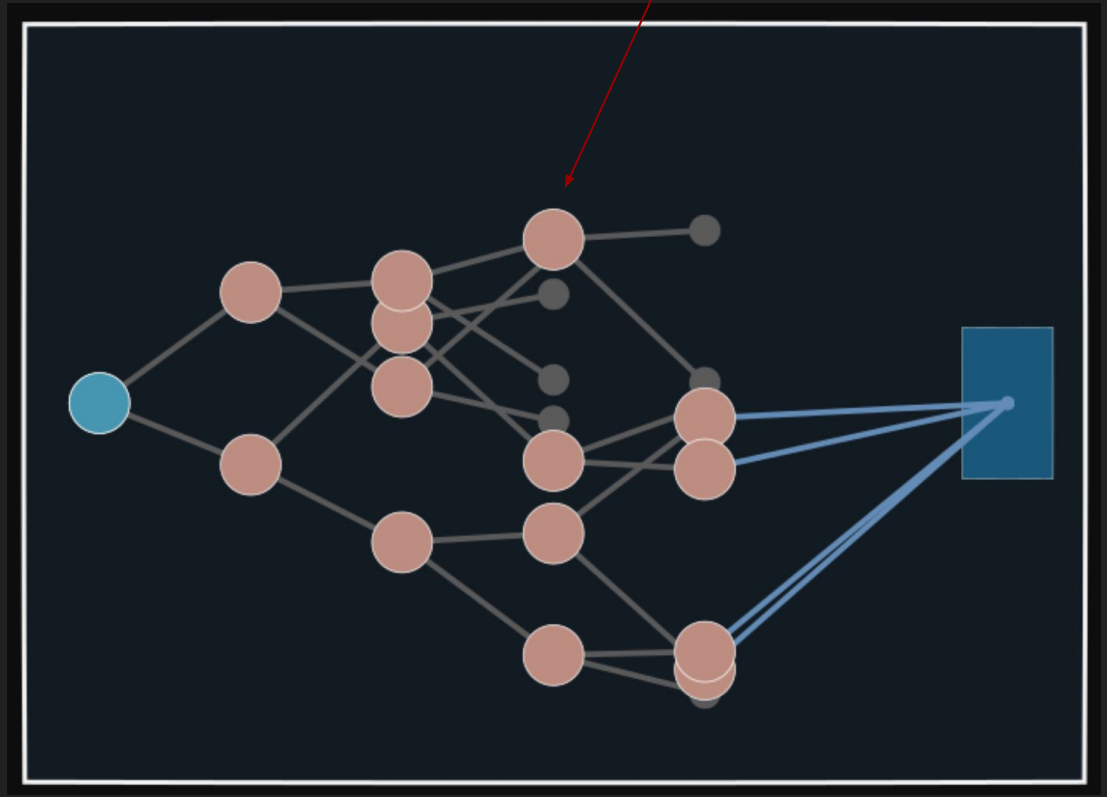
Combine the PRM with a *search algorithm.* Navigate the tree of reasoning steps. The PRM *guides* the generated CoTs.

1. Sample K next steps from current node
2. Use PRM to evaluate each possibility
3. Pick the next step(s) + proceed to the new node(s)

Instantiations: beam search, Monte Carlo Tree Search (MCTS).

PRM + Beam Search

Keep N = 4 “beams”.
Each proposes M = 2 next steps.
Top N from “frontier” are selected.
(From Sasha Rush’s slides.)



PRM + MCTS

Grow the CoT iteratively:

(Re)start at root. If you're *not* at leaf, move to child (based on exploration-exploitation).

If you're at leaf, you need to sample K children + do *MC rollout* for one of them.

Propagate to all previous nodes; the search tree has been extended.

How to learn PRM?

How to learn a PRM?

Lightman et al., 2023 (“Let’s Verify Step by Step”, OpenAI)

- Curate PRM800K, 800K *step-level* human feedback labels (via Scale)
- 75K solutions to 12K problems
- GPT-4 base model, MATH benchmark
- Train PRM to predict Pr(this step is correct)
- Finds PRM supervision > ORM supervision (if you do best-of-N at test-time…)
- (Note: for MATH, outcome verification is automatic)
- (Can unify generator + verifier LLMs: see Zhang et al., “Generative Verifiers…”, Google.)

Figure 1 from Lightman et al., 2023

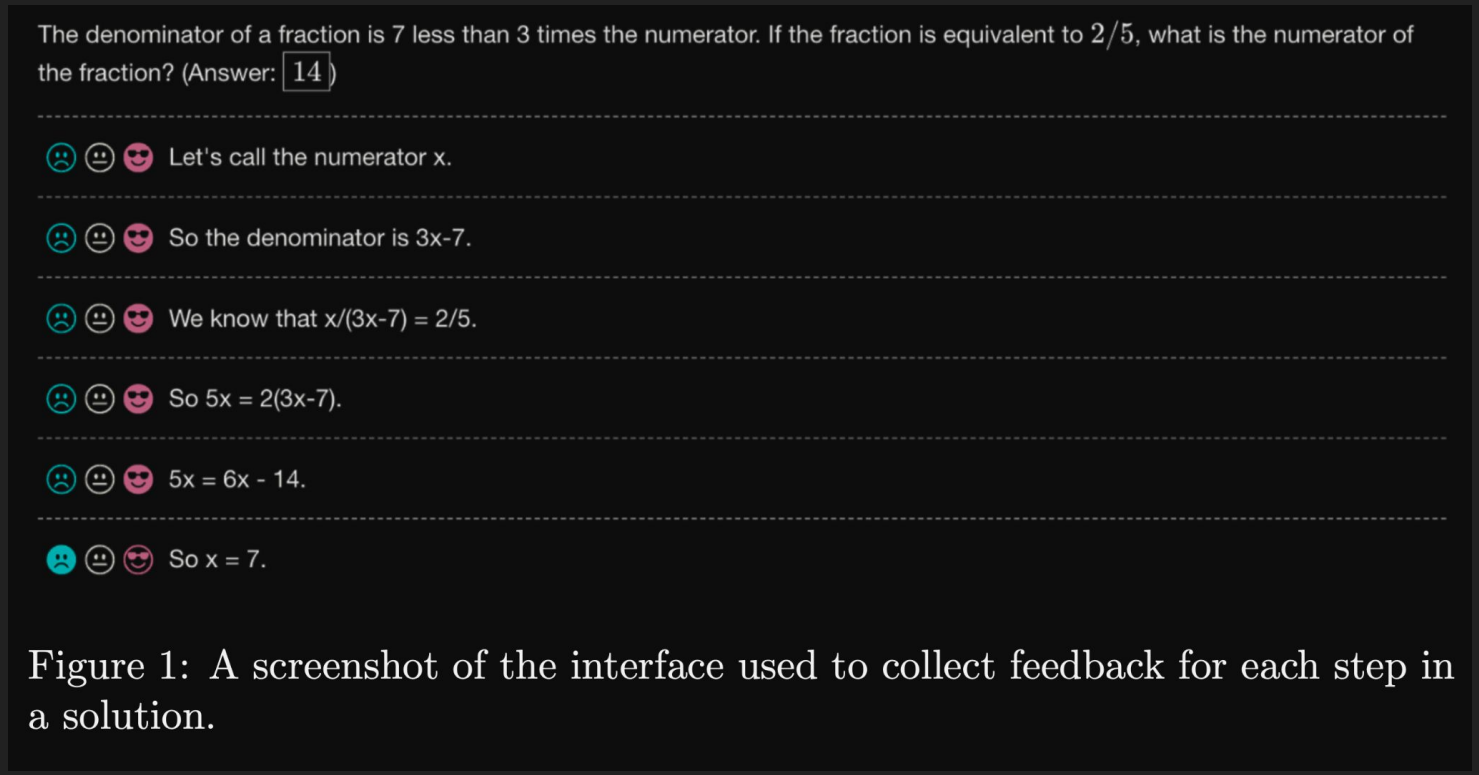


Figure 1: A screenshot of the interface used to collect feedback for each step in a solution.

How to learn PRM? (cont'd)

… But what if you don't have step-level human-annotated training data?

Wang et al. 2024 ("Math-Shepherd…", DeepSeek)

- Train PRM with *automatic* process supervision data
- Use *Monte Carlo rollouts* to evaluate the quality of individual steps
- To determine how good step i is,
  sample K completions (following the generator),
  & check how what proportion reach the correct answer.
- Issue: Monte Carlo rollouts are computationally expensive

Figure 2 from Wang et al., 2024

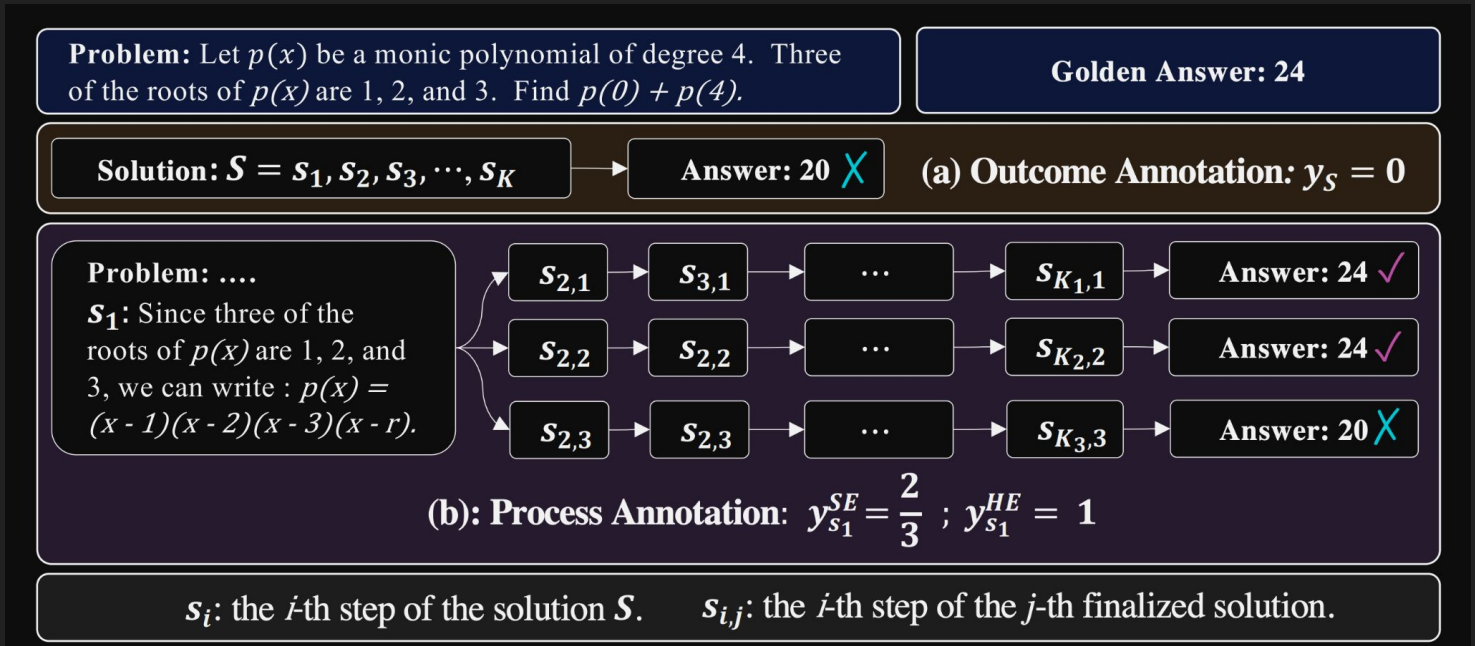


Figure 2: Comparison for previous automatic outcome annotation and our automatic process annotation. (a): automatic outcome annotation assigns a label to the entire solution S, dependent on the correctness of the answer; (b) automatic process annotation employs a 'completer' to finalize N

Learn RM + Improve generator

We can combine the previous two ideas as follows. The following is done at *train-time.*

Assume the RM is trained first (in any of the above ways).

At iteration k:

1. Sample “good” CoTs from current generator using ORM / PRM
2. Finetune generator on these CoTs

(The RM can be updated after each iteration, but this is not always necessary.)

Learn RM + Improve generator

Uesato et al., 2022 (“Solving math word problems with process and outcome-based feedback”, Google.)

- Compare ORM and PRM on GSM8K
- Train ORM as in Cobbe et al., 2021
- Train PRM using *human-annotated dataset* (size 1560) (all steps annotated)
- “ORM-RL”: best-of-N, collect “good” CoTs, finetune
- “PRM-RL”: run search guided by PRM, collect “good” CoTs, finetune
- (Known as “expert iteration”: distill the RM-guided “expert CoTs” into the generator via finetuning. Originally from Anthony et al., 2017 (“Thinking Fast and Slow…”))

Other papers also do RM-RL: for MCTS RM-RL, see Xie et al., 2024 (“Monte Carlo Tree Search Boosts Reasoning…”, Google)

Figure 2 from Uesato et al., 2022

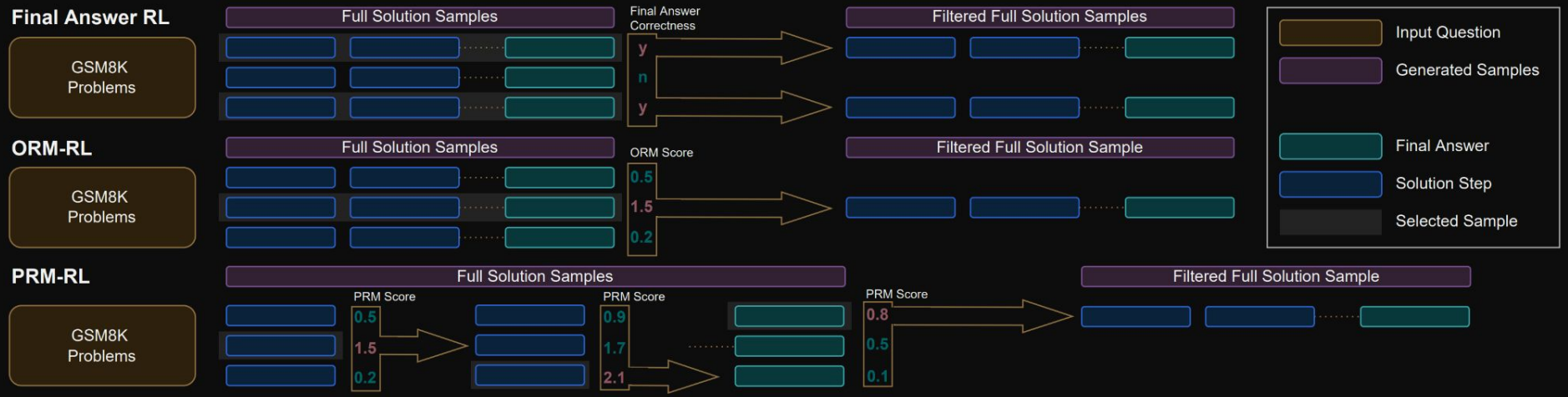


Figure 2 | **Policy improvement.** This schematic summarizes the different policy improvement operators for the Final-Answer RL, ORM-RL, and PRM-RL cases, which are described in the main text.

Papers mentioned

- Wei et al., 2022 (CoT)
- Singh et al., 2023 (ReST-EM)
- Shao et al., 2024 (GRPO)
- Liu et al., 2024 (RS-PO)
- Cobbe et al., 2021 (ORM)
- Sasha Rush's slides: https://srush.github.io/awesome-o1/o1-tutorial.pdf
- Lightman et al., 2023 (PRM from human annotations)
- Wang et al., 2024 (PRM via MC rollouts)
- Uesato et al., 2022 (ORM-RL, PRM-RL)
- Xie et al., 2024 (MCTS-based PRM-RL)
- Anthony et al., 2017 (Expert Iteration)
- DeepSeek-R1 Technical Report: https://github.com/deepseek-ai/DeepSeek-R1/blob/main/DeepSeek_R1.pdf

Discussion

- Other methods, relation to other LLM research (jailbreaking), follow-up RG talks (o3?)

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a+x}} = x$ is equal to
Response: <think> To solve the equation $\sqrt{a - \sqrt{a+x}} = x$, let's start by squaring both $\cdots$ $\left(\sqrt{a - \sqrt{a+x}}\right)^2 = x^2 \implies a - \sqrt{a+x} = x^2$. Rearrange to isolate the inner square root term: $(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$ $\ldots$ Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$ We started with the equation: $\sqrt{a - \sqrt{a+x}} = x$ First, let's square both sides: $a - \sqrt{a+x} = x^2 \implies \sqrt{a+x} = a - x^2$ Next, I could square both sides again, treating the equation: $\cdots$ $\ldots$

Table 3 | An interesting "aha moment" of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

4.2. Unsuccessful Attempts

In the early stages of developing DeepSeek-R1, we also encountered failures and setbacks along the way. We share our failure experiences here to provide insights, but this does not imply that these approaches are incapable of developing effective reasoning models.

Process Reward Model (PRM) PRM is a reasonable method to guide the model toward better approaches for solving reasoning tasks (Lightman et al., 2023; Uesato et al., 2022; Wang et al., 2023). However, in practice, PRM has three main limitations that may hinder its ultimate success. First, it is challenging to explicitly define a fine-grain step in general reasoning. Second, determining whether the current intermediate step is correct is a challenging task. Automated annotation using models may not yield satisfactory results, while manual annotation is not conducive to scaling up. Third, once a model-based PRM is introduced, it inevitably leads to reward hacking (Gao et al., 2022), and retraining the reward model needs additional training resources and it complicates the whole training pipeline. In conclusion, while PRM demonstrates a good ability to rerank the top-N responses generated by the model or assist in guided search (Snell et al., 2024), its advantages are limited compared to the additional computational overhead it introduces during large-scale reinforcement learning process in our experiments.

Monte Carlo Tree Search (MCTS) Inspired by AlphaGo (Silver et al., 2017b) and AlphaZero (Silver et al., 2017a), we explored using Monte Carlo Tree Search (MCTS) to enhance test-time compute scalability. This approach involves breaking answers into smaller parts to allow the model to explore the solution space systematically. To facilitate this, we prompt the model to generate multiple tags that correspond to specific reasoning steps necessary for the search. For training, we first use collected prompts to find answers via MCTS guided by a pre-trained value model. Subsequently, we use the resulting question-answer pairs to train both the actor model and the value model, iteratively refining the process.

However, this approach encounters several challenges when scaling up the training. First, unlike chess, where the search space is relatively well-defined, token generation presents an exponentially larger search space. To address this, we set a maximum extension limit for each node, but this can lead to the model getting stuck in local optima. Second, the value model directly influences the quality of generation since it guides each step of the search process. Training a fine-grained value model is inherently difficult, which makes it challenging for the model to iteratively improve. While AlphaGo's core success relied on training a value model to progressively enhance its performance, this principle proves difficult to replicate in our setup due to the complexities of token generation.